

PHÂN TÍCH VÀ TRIỂN KHAI SECURE BOOT TRÊN NỀN TẢNG ARM

1. Tổng quan Dự án

1.1. Giới thiệu

Trong các hệ thống nhúng hiện đại, đặc biệt là các thiết bị IoT, thiết bị công nghiệp và thiết bị mạng, việc đảm bảo tính toàn vẹn và xác thực của firmware là yêu cầu bắt buộc nhằm ngăn chặn các cuộc tấn công sửa đổi phần mềm trái phép. Secure Boot là cơ chế bảo mật được thiết kế để đảm bảo chỉ những thành phần phần mềm đã được xác thực mới được phép thực thi trong quá trình khởi động hệ thống.

Dự án tập trung nghiên cứu nguyên lý hoạt động của Secure Boot trên nền tảng ARM và triển khai một chuỗi khởi động an toàn (Chain of Trust) trên bo mạch ARM sử dụng U-Boot và Linux Embedded. Hệ thống sẽ thực hiện xác thực chữ ký số cho bootloader, kernel và device tree trước khi cho phép khởi động hệ điều hành.

Thông qua dự án, người thực hiện sẽ hiểu rõ kiến trúc boot sequence của ARM SoC, cơ chế xác thực RSA/ECDSA, quản lý khóa bảo mật và các kỹ thuật chống giả mạo firmware trong hệ thống nhúng.

1.2. Mục tiêu Dự án

Mục tiêu Kỹ thuật

- Nghiên cứu kiến trúc khởi động của ARM SoC.
- Phân tích quy trình Secure Boot và Chain of Trust.
- Tìm hiểu cơ chế ký số firmware bằng RSA hoặc ECDSA.
- Xây dựng môi trường phát triển Embedded Linux.
- Triển khai U-Boot Secure Boot.
- Tạo và quản lý cặp khóa Public/Private Key.
- Ký số Linux Kernel và Device Tree.
- Xác thực chữ ký trước khi boot hệ điều hành.
- Thử nghiệm các kịch bản tấn công thay đổi firmware.
- Đánh giá hiệu quả bảo vệ của Secure Boot.

Mục tiêu Chất lượng

- Hệ thống chỉ cho phép firmware hợp lệ được thực thi.

- Phát hiện chính xác firmware bị chỉnh sửa.
- Đảm bảo thời gian khởi động tăng không quá 10%.
- Xây dựng tài liệu triển khai đầy đủ và có khả năng tái sử dụng.
- Đảm bảo tính ổn định của hệ thống sau khi tích hợp Secure Boot.

1.3. Phạm vi Dự án

Phần cứng:

- Raspberry Pi 4 hoặc BeagleBone Black
- Máy tính phát triển Ubuntu Linux

Phần mềm:

- ARM Trusted Firmware (ATF)
- U-Boot Bootloader
- Linux Kernel
- OpenSSL
- Buildroot hoặc Yocto

Chức năng:

- Ký số bootloader
- Ký số kernel image
- Ký số device tree
- Xác thực chữ ký khi khởi động
- Kiểm tra tính toàn vẹn firmware

Ngoài phạm vi (Out-of-Scope)

- Full Disk Encryption
- Secure Element Hardware
- TPM Integration
- OTA Update Security
- Trusted Execution Environment (TEE)

2. Kiến trúc hệ thống

Hệ thống Secure Boot được xây dựng dựa trên mô hình **Chain of Trust (Chuỗi Tin Cậy)**, trong đó mỗi thành phần trong quá trình khởi động sẽ xác thực tính toàn vẹn và nguồn gốc của

thành phần kế tiếp trước khi chuyển quyền thực thi. Chuỗi tin cậy bắt đầu từ phần cứng (Root of Trust) và kéo dài đến khi hệ điều hành Linux được khởi động hoàn chỉnh.

2.1. Root of Trust (RoT)

Root of Trust là thành phần khởi đầu của chuỗi tin cậy, thường được tích hợp trong Boot ROM của SoC ARM. Thành phần này chứa khóa công khai (Public Key) hoặc giá trị hash được lưu trữ cố định trong phần cứng.

Nhiệm vụ của Root of Trust:

- Khởi tạo hệ thống.
- Đọc firmware khởi động từ bộ nhớ.
- Xác thực chữ ký số của firmware.
- Ngăn chặn việc thực thi firmware không hợp lệ.

2.2. ARM Trusted Firmware

ARM Trusted Firmware (ATF) đóng vai trò trung gian giữa Boot ROM và Bootloader.

Nhiệm vụ:

- Khởi tạo tài nguyên phần cứng cơ bản.
- Thiết lập môi trường bảo mật.
- Xác thực tính toàn vẹn của U-Boot.
- Duy trì chuỗi tin cậy trong quá trình khởi động.

2.3. U-Boot Bootloader

U-Boot là bootloader chính của hệ thống Embedded Linux.

Nhiệm vụ:

- Tải Linux Kernel.
- Tải Device Tree Blob (DTB).
- Xác thực chữ ký số của Kernel và DTB thông qua khóa công khai đã được nhúng trong U-Boot.
- Chỉ cho phép khởi động khi quá trình xác thực thành công.

2.4. Linux Kernel

Linux Kernel là thành phần điều khiển toàn bộ hệ thống sau khi quá trình Secure Boot hoàn tất.

Kernel chỉ được thực thi khi:

- Chữ ký số hợp lệ.
- Nội dung firmware không bị thay đổi.
- Được xác thực bởi U-Boot.

2.5. Root File System

Root File System chứa các thư viện, ứng dụng và dịch vụ của hệ điều hành.

Trong phạm vi dự án, RootFS được giả định là đáng tin cậy sau khi Kernel được xác thực thành công.

2.6. Chuỗi tin cậy (Chain of Trust)

Quá trình xác thực được thực hiện theo trình tự:

1. Boot ROM xác thực ARM Trusted Firmware.
2. ARM Trusted Firmware xác thực U-Boot.
3. U-Boot xác thực Linux Kernel.
4. U-Boot xác thực Device Tree.
5. Linux Kernel khởi động hệ điều hành.

Nếu bất kỳ bước xác thực nào thất bại, hệ thống sẽ dừng khởi động và thông báo lỗi bảo mật.

2.7. Lợi ích của kiến trúc

- Ngăn chặn firmware giả mạo.
- Phát hiện các thay đổi trái phép trên hệ thống.
- Đảm bảo tính toàn vẹn của chuỗi khởi động.
- Tăng cường khả năng chống tấn công vào bootloader và kernel.
- Đáp ứng các yêu cầu bảo mật trong hệ thống Embedded Linux hiện đại.

3. Công nghệ sử dụng

Mục tiêu của dự án là xây dựng và đánh giá cơ chế Secure Boot trên nền tảng ARM Embedded Linux. Hệ thống được cấu thành từ nhiều thành phần phối hợp với nhau để hình thành chuỗi tin cậy (Chain of Trust), đảm bảo chỉ những firmware hợp lệ mới được phép thực thi.

3.1. Hệ thống Quản lý Khóa

Hệ thống Secure Boot sử dụng cơ chế mật mã khóa công khai (Public Key Cryptography).

Private Key

Chức năng:

- Ký số firmware.
- Chỉ tồn tại trên máy phát triển.

Yêu cầu:

- Không được lưu trên thiết bị đích.
- Được bảo vệ nghiêm ngặt.

Public Key

Chức năng:

- Xác thực chữ ký số.
- Được nhúng vào U-Boot hoặc Firmware.

Public Key là nền tảng để thiết lập Chain of Trust trong hệ thống.

3.2. Công nghệ Mật mã

RSA

Thuật toán bất đối xứng được sử dụng để ký và xác thực firmware.

Cấu hình sử dụng:

- RSA-2048.
- RSA-4096.

SHA-256

Thuật toán băm được sử dụng để:

- Tạo giá trị hash của firmware.
- Phát hiện thay đổi dữ liệu.

FIT Image

Flattened Image Tree (FIT) là định dạng image được U-Boot hỗ trợ.

Ưu điểm:

- Hỗ trợ ký số.
- Hỗ trợ nhiều firmware trong một image.
- Dễ dàng mở rộng Secure Boot.

3.3. Công cụ Phát triển

Yocto

Công cụ xây dựng hệ thống Embedded Linux tối giản.

Chức năng:

- Build Toolchain.
- Build Kernel.
- Build Root Filesystem.

OpenSSL

Thư viện mật mã dùng để:

- Tạo khóa RSA.
- Ký firmware.
- Kiểm tra chữ ký số.

U-Boot Tools

Bao gồm:

- mkimage.
- dumpimage.

Chức năng:

- Tạo FIT Image.
- Tích hợp chữ ký số vào firmware.

4. Yêu cầu Hệ thống

4.1. Yêu cầu Chức năng (Functional Requirements)

FR-01: Xác thực Bootloader

Hệ thống phải kiểm tra tính hợp lệ của Bootloader trước khi cho phép thực thi.

Mô tả:

- Sử dụng khóa công khai đã được nhúng trong hệ thống.
- Kiểm tra chữ ký số của Bootloader.
- Dừng quá trình khởi động nếu xác thực thất bại.

FR-02: Xác thực Linux Kernel

Hệ thống phải xác thực Linux Kernel trước khi nạp vào bộ nhớ và thực thi.

Mô tả:

- Kiểm tra chữ ký số của Kernel Image.
- Đảm bảo Kernel không bị chỉnh sửa sau khi được ký.
- Chỉ cho phép khởi động Kernel hợp lệ.

FR-03: Xác thực Device Tree Blob (DTB)

Hệ thống phải xác thực tệp Device Tree trước khi truyền cho Kernel.

Mô tả:

- Kiểm tra chữ ký số của DTB.
- Phát hiện các thay đổi trái phép trong cấu hình phần cứng.

FR-04: Kiểm tra tính toàn vẹn Firmware

Hệ thống phải phát hiện mọi thay đổi đối với firmware.

Mô tả:

- Sử dụng thuật toán băm SHA-256.
- So sánh giá trị hash với chữ ký số đã được xác thực.

FR-05: Từ chối Firmware không hợp lệ

Hệ thống phải ngăn chặn việc khởi động nếu firmware không vượt qua quá trình xác thực.

Mô tả:

- Dừng quá trình boot.
- Hiển thị hoặc ghi nhận lỗi bảo mật.
- Không chuyển quyền thực thi cho firmware bị lỗi.

FR-06: Quản lý Khóa Xác thực

Hệ thống phải hỗ trợ cơ chế sử dụng cặp khóa bất đối xứng.

Mô tả:

- Public Key được nhúng vào Bootloader.
- Private Key chỉ được sử dụng trong môi trường phát triển để ký firmware.

FR-07: Hỗ trợ Firmware đã Ký số

Hệ thống phải hỗ trợ việc tạo và sử dụng firmware được ký số.

Mô tả:

- Hỗ trợ FIT Image của U-Boot.
- Hỗ trợ chữ ký RSA.
- Hỗ trợ tích hợp nhiều thành phần firmware trong một image.

FR-08: Ghi nhận Thông tin Kiểm thử

Hệ thống phải cung cấp thông tin phục vụ việc đánh giá Secure Boot.

Mô tả:

- Hiển thị trạng thái xác thực.
- Ghi log quá trình boot thông qua UART.
- Hỗ trợ phân tích lỗi trong quá trình xác thực.

4.2. Yêu cầu Phi chức năng (Non-Functional Requirements)

NFR-01: Bảo mật

Hệ thống phải đảm bảo tính bảo mật của chuỗi khởi động.

Yêu cầu:

- Chỉ cho phép firmware đã được ký số hợp lệ được thực thi.
- Không lưu Private Key trên thiết bị.
- Ngăn chặn việc thay đổi firmware trái phép.

NFR-02: Hiệu năng

Việc triển khai Secure Boot không được ảnh hưởng đáng kể đến thời gian khởi động.

Yêu cầu:

- Thời gian xác thực firmware không vượt quá 3 giây.
- Thời gian boot tăng không quá 10% so với hệ thống không sử dụng Secure Boot.

NFR-03: Độ tin cậy

Hệ thống phải hoạt động ổn định trong các điều kiện sử dụng bình thường.

Yêu cầu:

- Tỷ lệ xác thực thành công đạt 100% đối với firmware hợp lệ.
- Không xảy ra lỗi boot do cơ chế xác thực.

NFR-04: Khả năng bảo trì

Hệ thống phải dễ dàng nâng cấp và mở rộng.

Yêu cầu:

- Hỗ trợ cập nhật firmware mới.
- Hỗ trợ thay đổi khóa xác thực khi cần thiết.
- Tài liệu triển khai đầy đủ và dễ sử dụng.

NFR-05: Khả năng mở rộng

Hệ thống phải có khả năng tích hợp với các cơ chế bảo mật nâng cao trong tương lai.

Yêu cầu:

- Hỗ trợ ARM Trusted Firmware.
- Hỗ trợ Verified Boot.
- Hỗ trợ Secure Firmware Update.
- Hỗ trợ Trusted Execution Environment (TEE).

NFR-06: Khả năng kiểm thử

Hệ thống phải cho phép đánh giá hiệu quả của Secure Boot.

Yêu cầu:

- Hỗ trợ các kịch bản giả lập tấn công.
- Hỗ trợ thay đổi firmware để kiểm thử tính toàn vẹn.
- Cung cấp log phục vụ phân tích kết quả.

Tiêu chí Đánh giá Thành công

- Dự án được xem là thành công khi đáp ứng các điều kiện sau:
- Secure Boot được triển khai thành công trên nền tảng ARM.
- Firmware hợp lệ được khởi động bình thường.
- Firmware bị thay đổi bị từ chối thực thi.
- Chuỗi tin cậy được duy trì trong toàn bộ quá trình khởi động.
- Hệ thống hoạt động ổn định và đáp ứng các yêu cầu hiệu năng đã đề ra.

5. Demo và Tài liệu Bàn giao

5.1. Demo Hệ thống

Mục tiêu của phần demo là chứng minh cơ chế Secure Boot đã được triển khai thành công trên nền tảng ARM và hoạt động đúng theo thiết kế.

Demo 1: Khởi động với Firmware Hợp lệ

Mô tả:

- Linux Kernel được ký số bằng Private Key.
- Device Tree được ký số hợp lệ.
- U-Boot chứa Public Key tương ứng.

Kết quả mong đợi:

- U-Boot xác thực thành công chữ ký số.
- Kernel được nạp vào bộ nhớ.
- Hệ điều hành Linux khởi động bình thường.

Minh chứng:

- Log UART hiển thị trạng thái xác thực thành công.

- Hệ thống hiển thị màn hình đăng nhập Linux.

Demo 2: Phát hiện Firmware Bị Chỉnh sửa

Mô tả:

- Thay đổi một phần nội dung Linux Kernel sau khi đã ký số.
- Không thực hiện ký lại firmware.

Kết quả mong đợi:

- U-Boot phát hiện chữ ký không hợp lệ.
- Quá trình boot bị dừng.
- Thông báo lỗi xác thực được ghi nhận trên UART.

Minh chứng:

- Bad Data Hash
- Signature Verification Failed
- Boot Aborted

Demo 3: Sử dụng Sai Khóa Xác thực

Mô tả:

- Ký firmware bằng một Private Key khác.
- Public Key trong U-Boot không tương ứng.

Kết quả mong đợi:

- Chữ ký số không được xác thực.
- Hệ thống từ chối khởi động.

Minh chứng:

- Verification Failed
- Invalid Signature

Demo 4: Đánh giá Hiệu năng

Các chỉ số đánh giá:

- Thời gian boot trước khi tích hợp Secure Boot.
- Thời gian boot sau khi tích hợp Secure Boot.
- Thời gian xác thực Kernel.
- Thời gian xác thực Device Tree.

Mục tiêu:

- Chứng minh cơ chế Secure Boot không ảnh hưởng đáng kể đến hiệu năng hệ thống.

5.2. Kết quả Đạt được

Sau khi hoàn thành dự án, hệ thống đạt được các kết quả sau:

- Xây dựng thành công môi trường Embedded Linux trên nền tảng ARM.
- Tích hợp Secure Boot vào U-Boot.
- Tạo và quản lý cặp khóa RSA phục vụ xác thực firmware.
- Ký số Linux Kernel và Device Tree.
- Xác thực thành công firmware trong quá trình khởi động.
- Phát hiện và ngăn chặn firmware bị sửa đổi.
- Xây dựng chuỗi tin cậy (Chain of Trust) từ Bootloader đến Kernel.

5.3. Tài liệu Bàn giao

Tài liệu Thiết kế

- Software Requirement Specification (SRS)
- High-Level Design (HLD)
- Detailed Design Document (DDD)
- Secure Boot Architecture Document

Mục đích:

- Mô tả yêu cầu, kiến trúc và thiết kế của hệ thống.

Tài liệu Triển khai

- Hướng dẫn Build U-Boot
- Hướng dẫn Build Linux Kernel
- Hướng dẫn tạo FIT Image
- Hướng dẫn ký số Firmware
- Hướng dẫn cấu hình Secure Boot

Mục đích:

- Hỗ trợ tái triển khai hệ thống trên nền tảng ARM khác.

Tài liệu Kiểm thử

Bao gồm:

- Test Plan
- Test Case

- Test Report
- Security Validation Report

Nội dung:

- Kết quả kiểm thử chức năng.
- Kết quả kiểm thử bảo mật.
- Kết quả đánh giá hiệu năng.

Tài liệu Hướng dẫn Sử dụng

Bao gồm:

- Hướng dẫn khởi động hệ thống.
- Hướng dẫn cập nhật firmware.
- Hướng dẫn kiểm tra trạng thái Secure Boot.
- Hướng dẫn xử lý lỗi thường gặp.

5.4. Mã nguồn Bàn giao

Cấu trúc mã nguồn:

secure-boot-project/

```
|
|— docs/
|   |— HLD.pdf
|   |— DDD.pdf
|   └─ TestReport.pdf
|
|— bootloader/
|   └─ u-boot/
|
|— kernel/
|   └─ linux/
|
|— keys/
|   |— public.key
|   └─ private.key
```

```
|
|— scripts/
|   |— sign_image.sh
|   |— verify_image.sh
|   └— build_fit.sh
|
|— images/
|   |— Image
|   |— fitImage
|   └— board.dtb
|
└— README.md
```

5.5. Sản phẩm Bàn giao Cuối cùng

Dự án hoàn thành sẽ bàn giao các thành phần sau:

Phần mềm

- U-Boot tích hợp Secure Boot.
- Linux Kernel đã được ký số.
- FIT Image hoàn chỉnh.
- Bộ script ký số firmware.

Tài liệu

- Báo cáo kỹ thuật.
- Tài liệu triển khai.
- Tài liệu kiểm thử.
- Hướng dẫn sử dụng.

Demo

- Video trình diễn Secure Boot.
- Video kiểm thử firmware bị chỉnh sửa.
- Log xác thực trong quá trình khởi động.

Kết quả cuối cùng

Một hệ thống Embedded Linux trên nền tảng ARM có khả năng xác thực firmware trong quá trình khởi động, đảm bảo tính toàn vẹn và xác thực của hệ thống thông qua cơ chế Secure Boot.